

PRODIGY_DS_02

March 16, 2026

1 EDA (Exploratory Data Analysis) & Processing

```
[116]: #Basic EDA
#Handling null values/missing data
#Handling Outliers
#handling skewness
#handling categorical features
#Data Normalization And scaling
#feature engineering
```

```
[117]: # import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
[118]: df=pd.read_csv('cars.csv')
```

2 Basic EDA

It involves info,describe,descriptive statistics of the columns

```
[119]: # first 5 rows
df.head()
```

```
[119]:
```

	symboling	normalized-losses	make	fuel-type	body-style	\
0	3	?	alfa-romero	gas	convertible	
1	3	?	alfa-romero	gas	convertible	
2	1	?	alfa-romero	gas	hatchback	
3	2	164	audi	gas	sedan	
4	2	164	audi	gas	sedan	

	drive-wheels	engine-location	width	height	engine-type	engine-size	\
0	rwd	front	64.1	48.8	dohc	130	
1	rwd	front	64.1	48.8	dohc	130	

2	rwd	front	65.5	52.4	ohcv	152
3	fwd	front	66.2	54.3	ohc	109
4	4wd	front	66.4	54.3	ohc	136

	horsepower	city-mpg	highway-mpg	price
0	111	21	27	13495
1	111	21	27	16500
2	154	19	26	16500
3	102	24	30	13950
4	115	18	22	17450

```
[120]: #Shape of Dataframe
df.shape
```

```
[120]: (205, 15)
```

```
[121]: #Last 5 rows
df.tail()
```

```
[121]:      symboling  normalized-losses  make fuel-type body-style drive-wheels \
200         -1             95  volvo      gas      sedan        rwd
201         -1             95  volvo      gas      sedan        rwd
202         -1             95  volvo      gas      sedan        rwd
203         -1             95  volvo    diesel      sedan        rwd
204         -1             95  volvo      gas      sedan        rwd
```

	engine-location	width	height	engine-type	engine-size	horsepower	\
200	front	68.9	55.5	ohc	141	114	
201	front	68.8	55.5	ohc	141	160	
202	front	68.9	55.5	ohcv	173	134	
203	front	68.9	55.5	ohc	145	106	
204	front	68.9	55.5	ohc	141	114	

	city-mpg	highway-mpg	price
200	23	28	16845
201	19	25	19045
202	18	23	21485
203	26	27	22470
204	19	25	22625

```
[122]: #name of columns
df.columns
```

```
[122]: Index(['symboling', 'normalized-losses', 'make', 'fuel-type', 'body-style',
        'drive-wheels', 'engine-location', 'width', 'height', 'engine-type',
        'engine-size', 'horsepower', 'city-mpg', 'highway-mpg', 'price'],
        dtype='object')
```

```
[123]: # Datatypes of all columns
df.dtypes
```

```
[123]: symboling          int64
normalized-losses      object
make                   object
fuel-type              object
body-style             object
drive-wheels           object
engine-location        object
width                  float64
height                 float64
engine-type            object
engine-size            int64
horsepower             object
city-mpg               int64
highway-mpg            int64
price                  int64
dtype: object
```

```
[124]: #Detailed information of the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   symboling              205 non-null   int64
1   normalized-losses      205 non-null   object
2   make                   205 non-null   object
3   fuel-type              205 non-null   object
4   body-style             205 non-null   object
5   drive-wheels           205 non-null   object
6   engine-location        205 non-null   object
7   width                  205 non-null   float64
8   height                 205 non-null   float64
9   engine-type            205 non-null   object
10  engine-size            205 non-null   int64
11  horsepower             205 non-null   object
12  city-mpg               205 non-null   int64
13  highway-mpg            205 non-null   int64
14  price                  205 non-null   int64
dtypes: float64(2), int64(5), object(8)
memory usage: 24.2+ KB
```

```
[125]: #Descriptive Statistics of numeric column
df.describe()
```

```
[125]:      symboling      width      height  engine-size  city-mpg  \
count  205.000000  205.000000  205.000000  205.000000  205.000000
mean    0.834146   65.907805   53.724878   126.907317   25.219512
std     1.245307    2.145204    2.443522    41.642693    6.542142
min    -2.000000   60.300000   47.800000    61.000000   13.000000
25%     0.000000   64.100000   52.000000    97.000000   19.000000
50%     1.000000   65.500000   54.100000   120.000000   24.000000
75%     2.000000   66.900000   55.500000   141.000000   30.000000
max      3.000000   72.300000   59.800000   326.000000   49.000000
```

```
      highway-mpg      price
count  205.000000  205.000000
mean    30.751220  13227.478049
std      6.886443   7902.651615
min     16.000000   5118.000000
25%     25.000000   7788.000000
50%     30.000000  10345.000000
75%     34.000000  16500.000000
max     54.000000  45400.000000
```

```
[126]: #descriptive stats for categorical columns
df.describe(include=['object'])
```

```
[126]:      normalized-losses  make fuel-type body-style drive-wheels  \
count                205    205      205      205      205
unique                52     22        2        5        3
top                   ?  toyota    gas    sedan    fwd
freq                 41     32    185     96    120
```

```
      engine-location engine-type horsepower
count                205      205      205
unique                2        7       60
top                 front    ohc      68
freq                202    148      19
```

```
[127]: #All columns
df.describe(include='all')
```

```
[127]:      symboling  normalized-losses  make fuel-type body-style  \
count  205.000000                205    205      205      205
unique      NaN                52     22        2        5
top        NaN                ?  toyota    gas    sedan
freq      NaN                41     32    185     96
mean    0.834146                NaN    NaN    NaN    NaN
std     1.245307                NaN    NaN    NaN    NaN
min    -2.000000                NaN    NaN    NaN    NaN
25%     0.000000                NaN    NaN    NaN    NaN
```

50%	1.000000	NaN	NaN	NaN	NaN
75%	2.000000	NaN	NaN	NaN	NaN
max	3.000000	NaN	NaN	NaN	NaN

	drive-wheels	engine-location	width	height	engine-type \
count	205	205	205.000000	205.000000	205
unique	3	2	NaN	NaN	7
top	fwd	front	NaN	NaN	ohc
freq	120	202	NaN	NaN	148
mean	NaN	NaN	65.907805	53.724878	NaN
std	NaN	NaN	2.145204	2.443522	NaN
min	NaN	NaN	60.300000	47.800000	NaN
25%	NaN	NaN	64.100000	52.000000	NaN
50%	NaN	NaN	65.500000	54.100000	NaN
75%	NaN	NaN	66.900000	55.500000	NaN
max	NaN	NaN	72.300000	59.800000	NaN

	engine-size	horsepower	city-mpg	highway-mpg	price
count	205.000000	205	205.000000	205.000000	205.000000
unique	NaN	60	NaN	NaN	NaN
top	NaN	68	NaN	NaN	NaN
freq	NaN	19	NaN	NaN	NaN
mean	126.907317	NaN	25.219512	30.751220	13227.478049
std	41.642693	NaN	6.542142	6.886443	7902.651615
min	61.000000	NaN	13.000000	16.000000	5118.000000
25%	97.000000	NaN	19.000000	25.000000	7788.000000
50%	120.000000	NaN	24.000000	30.000000	10345.000000
75%	141.000000	NaN	30.000000	34.000000	16500.000000
max	326.000000	NaN	49.000000	54.000000	45400.000000

3 if we notice

1.normalized-losses & horsepower columns are containing numerical data but their datatype is showing them as categorical 2.Also in normalized-losses column top value is "?",what is that

```
[128]: df['normalized-losses'].unique()
```

```
[128]: array(['?', '164', '158', '192', '188', '121', '98', '81', '118', '148',
            '110', '145', '137', '101', '78', '106', '85', '107', '104', '113',
            '150', '129', '115', '93', '142', '161', '153', '125', '128',
            '122', '103', '168', '108', '194', '231', '119', '154', '74',
            '186', '83', '102', '89', '87', '77', '91', '134', '65', '197',
            '90', '94', '256', '95'], dtype=object)
```

```
[129]: df['normalized-losses'].value_counts()
```

[129]: normalized-losses

?	41
161	11
91	8
150	7
128	6
104	6
134	6
74	5
103	5
168	5
95	5
102	5
94	5
65	5
85	5
93	4
106	4
118	4
122	4
148	4
154	3
83	3
125	3
115	3
137	3
101	3
194	2
197	2
188	2
164	2
158	2
192	2
110	2
145	2
113	2
81	2
89	2
87	2
108	2
119	2
153	2
129	2
98	1
121	1
107	1
78	1

```

231      1
142      1
77       1
186      1
90       1
256      1
Name: count, dtype: int64

```

```
[130]: df['horsepower'].unique()
```

```
[130]: array(['111', '154', '102', '115', '110', '140', '160', '101', '121',
            '182', '48', '70', '68', '88', '145', '58', '76', '60', '86',
            '100', '78', '90', '176', '262', '135', '84', '64', '120', '72',
            '123', '155', '184', '175', '116', '69', '55', '97', '152', '200',
            '95', '142', '143', '207', '288', '?', '73', '82', '94', '62',
            '56', '112', '92', '161', '156', '52', '85', '114', '162', '134',
            '106'], dtype=object)
```

```
[131]: df['horsepower'].value_counts()
```

```
[131]: horsepower
68      19
70      11
69      10
116      9
110      8
95       7
88       6
114      6
62       6
101      6
160      6
102      5
76       5
97       5
82       5
145      5
84       5
123      4
86       4
92       4
111      4
121      3
90       3
182      3
85       3
73       3

```

```

152      3
207      3
112      2
56       2
156      2
161      2
52       2
162      2
?        2
155      2
100      2
94       2
184      2
176      2
115      1
154      1
140      1
48       1
120      1
64       1
135      1
262      1
78       1
60       1
58       1
55       1
288      1
143      1
142      1
200      1
175      1
72       1
134      1
106      1
Name: count, dtype: int64

```

3.1 Handling Null values

To handle null/missing values we can do 2 things 1. Drop those null/missing values 2. Fill those null/missing values

```
[132]: df['normalized-losses'].replace('?', np.nan, inplace=True)
```

```
[133]: df['horsepower'].replace('?', np.nan, inplace=True)
```

4 Now we will convert the datatypes of these columns to float datatype

```
[134]: df['normalized-losses']=df['normalized-losses'].astype('float64')
```

```
[135]: df['horsepower']=df['horsepower'].astype('float64')
```

```
[136]: df.dtypes
```

```
[136]: symboling          int64
normalized-losses      float64
make                   object
fuel-type              object
body-style             object
drive-wheels           object
engine-location        object
width                  float64
height                 float64
engine-type            object
engine-size            int64
horsepower             float64
city-mpg               int64
highway-mpg            int64
price                  int64
dtype: object
```

```
[137]: df['normalized-losses'].unique()
```

```
[137]: array([ nan, 164., 158., 192., 188., 121.,  98.,  81., 118., 148., 110.,
        145., 137., 101.,  78., 106.,  85., 107., 104., 113., 150., 129.,
        115.,  93., 142., 161., 153., 125., 128., 122., 103., 168., 108.,
        194., 231., 119., 154.,  74., 186.,  83., 102.,  89.,  87.,  77.,
        91., 134.,  65., 197.,  90.,  94., 256.,  95.] )
```

```
[138]: df['horsepower'].unique()
```

```
[138]: array([111., 154., 102., 115., 110., 140., 160., 101., 121., 182.,  48.,
        70.,  68.,  88., 145.,  58.,  76.,  60.,  86., 100.,  78.,  90.,
        176., 262., 135.,  84.,  64., 120.,  72., 123., 155., 184., 175.,
        116.,  69.,  55.,  97., 152., 200.,  95., 142., 143., 207., 288.,
         nan,  73.,  82.,  94.,  62.,  56., 112.,  92., 161., 156.,  52.,
        85., 114., 162., 134., 106.] )
```

```
[139]: #let's create the copy of original df
df_copy=df.copy()
```

```
[140]: df_copy.shape
```

```
[140]: (205, 15)
```

```
[141]: df_copy.isnull()
```

```
[141]:
```

	symboling	normalized-losses	make	fuel-type	body-style	drive-wheels	\
0	False	True	False	False	False	False	
1	False	True	False	False	False	False	
2	False	True	False	False	False	False	
3	False	False	False	False	False	False	
4	False	False	False	False	False	False	
..	...	...	...	...	...	...	
200	False	False	False	False	False	False	
201	False	False	False	False	False	False	
202	False	False	False	False	False	False	
203	False	False	False	False	False	False	
204	False	False	False	False	False	False	

	engine-location	width	height	engine-type	engine-size	horsepower	\
0	False	False	False	False	False	False	
1	False	False	False	False	False	False	
2	False	False	False	False	False	False	
3	False	False	False	False	False	False	
4	False	False	False	False	False	False	
..	...	...	...	...	...	...	
200	False	False	False	False	False	False	
201	False	False	False	False	False	False	
202	False	False	False	False	False	False	
203	False	False	False	False	False	False	
204	False	False	False	False	False	False	

	city-mpg	highway-mpg	price
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False
..	...	...	...
200	False	False	False
201	False	False	False
202	False	False	False
203	False	False	False
204	False	False	False

```
[205 rows x 15 columns]
```

```
[142]: df_copy.isnull().sum()
```

```
[142]: symboling          0
      normalized-losses  41
      make              0
      fuel-type         0
      body-style        0
      drive-wheels      0
      engine-location    0
      width             0
      height            0
      engine-type       0
      engine-size       0
      horsepower        2
      city-mpg          0
      highway-mpg       0
      price             0
      dtype: int64
```

```
[143]: df_copy.shape
```

```
[143]: (205, 15)
```

```
[144]: # drop all the rows
      df_copy.dropna(axis=0,inplace=True)
```

```
[145]: df_copy.shape
```

```
[145]: (164, 15)
```

```
[146]: #2nd Copy
      df_copy=df.copy()
```

```
[147]: df_copy.shape
```

```
[147]: (205, 15)
```

```
[148]: df_copy.isnull().sum()
```

```
[148]: symboling          0
      normalized-losses  41
      make              0
      fuel-type         0
      body-style        0
      drive-wheels      0
      engine-location    0
      width             0
      height            0
      engine-type       0
      engine-size       0
```

```
horsepower      2
city-mpg        0
highway-mpg     0
price           0
dtype: int64
```

```
[149]: #Drop columns
df_copy.dropna(axis=1,inplace=True)
```

```
[150]: df_copy.shape
```

```
[150]: (205, 13)
```

```
[151]: df_copy.isnull().sum()
```

```
[151]: symboling      0
make              0
fuel-type         0
body-style        0
drive-wheels      0
engine-location   0
width             0
height            0
engine-type       0
engine-size       0
city-mpg          0
highway-mpg       0
price             0
dtype: int64
```

5 Filling Null Values

1. Fill null values of numerical columns with either mean or median value
2. Fill null values of categorical columns with mode value

```
[152]: df.isnull()
```

```
[152]:
```

	symboling	normalized-losses	make	fuel-type	body-style	drive-wheels	\
0	False	True	False	False	False	False	
1	False	True	False	False	False	False	
2	False	True	False	False	False	False	
3	False	False	False	False	False	False	
4	False	False	False	False	False	False	
..	...	...	...	...	...	...	
200	False	False	False	False	False	False	
201	False	False	False	False	False	False	
202	False	False	False	False	False	False	

203	False	False	False	False	False	False
204	False	False	False	False	False	False

	engine-location	width	height	engine-type	engine-size	horsepower	\
0	False	False	False	False	False	False	
1	False	False	False	False	False	False	
2	False	False	False	False	False	False	
3	False	False	False	False	False	False	
4	False	False	False	False	False	False	
..	...	...	...	...	...	...	
200	False	False	False	False	False	False	
201	False	False	False	False	False	False	
202	False	False	False	False	False	False	
203	False	False	False	False	False	False	
204	False	False	False	False	False	False	

	city-mpg	highway-mpg	price
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False
..	...	...	...
200	False	False	False
201	False	False	False
202	False	False	False
203	False	False	False
204	False	False	False

[205 rows x 15 columns]

```
[153]: df.isnull().sum()
```

```
[153]: symboling          0
normalized-losses      41
make                   0
fuel-type              0
body-style             0
drive-wheels           0
engine-location        0
width                  0
height                 0
engine-type            0
engine-size            0
horsepower             2
city-mpg               0
highway-mpg            0
```

```
price          0
dtype: int64
```

```
[154]: normalized_mean=df['normalized-losses'].mean()
normalized_mean
```

```
[154]: np.float64(122.0)
```

```
[155]: horsepower_mean=df['horsepower'].mean()
horsepower_mean
```

```
[155]: np.float64(104.25615763546799)
```

```
[156]: df['normalized-losses'].fillna(normalized_mean,inplace=True)
```

```
[157]: df['horsepower'].fillna(horsepower_mean,inplace=True)
```

```
[158]: df.isnull().sum()
```

```
[158]: symboling          0
normalized-losses      0
make                   0
fuel-type              0
body-style             0
drive-wheels           0
engine-location        0
width                  0
height                 0
engine-type            0
engine-size            0
horsepower             0
city-mpg               0
highway-mpg            0
price                  0
dtype: int64
```

```
[159]: df['normalized-losses'].mean()
```

```
[159]: np.float64(122.0)
```

```
[160]: df['horsepower'].mean()
```

```
[160]: np.float64(104.25615763546797)
```

```
[161]: df_copy.isnull().sum()
```

```
[161]: symboling          0
make                   0
```

```

fuel-type      0
body-style     0
drive-wheels   0
engine-location 0
width          0
height         0
engine-type    0
engine-size    0
city-mpg       0
highway-mpg    0
price         0
dtype: int64

```

```
[162]: # let's add some null values in categorical columns
df_copy.loc[50:100, 'fuel-type']=np.nan
```

```
[163]: df_copy.isnull().sum()
```

```

[163]: symboling      0
make              0
fuel-type        51
body-style       0
drive-wheels     0
engine-location  0
width            0
height           0
engine-type      0
engine-size      0
city-mpg         0
highway-mpg      0
price            0
dtype: int64

```

```
[164]: fuel_mode=df_copy['fuel-type'].mode()
fuel_mode
```

```

[164]: 0    gas
Name: fuel-type, dtype: object

```

```
[165]: fuel_mode=df_copy['fuel-type'].mode()[0]
fuel_mode
```

```
[165]: 'gas'
```

```
[166]: df_copy['fuel-type'].value_counts()
```

```
[166]: fuel-type
      gas      141
      diesel   13
      Name: count, dtype: int64
```

```
[167]: #Replace Null values with mode
      df_copy['fuel-type'].fillna(fuel_mode,inplace=True)
```

```
[168]: df_copy.isnull().sum()
```

```
[168]: symboling      0
      make          0
      fuel-type      0
      body-style     0
      drive-wheels   0
      engine-location 0
      width          0
      height         0
      engine-type     0
      engine-size     0
      city-mpg        0
      highway-mpg     0
      price          0
      dtype: int64
```

```
[169]: df_copy['fuel-type'].value_counts()
```

```
[169]: fuel-type
      gas      192
      diesel   13
      Name: count, dtype: int64
```

6 Handling Outliers

How are we going to identify outliers - Z-score: here we measure how far the data point from the mean in term of standard deviation. If the value is ± 3 SD from the mean then we consider it as outlier - Interquartile Range (IQR): $IQR = Q3 - Q1$ Lower Bound = $Q1 - 1.5 * IQR$ Upper Bound = $Q3 + 1.5 * IQR$ Any data point that falls below the lower bound or above the upper bound is considered an outlier - Boxplot: graphical representation of the data that highlights median, quartiles, upper and lower whisker. any value that lies outside the whiskers is considered an outlier

7 Handling Outliers

1. these outliers can be dropped
2. these outliers can be replaced by mean/median

```
[170]: df.describe()
```

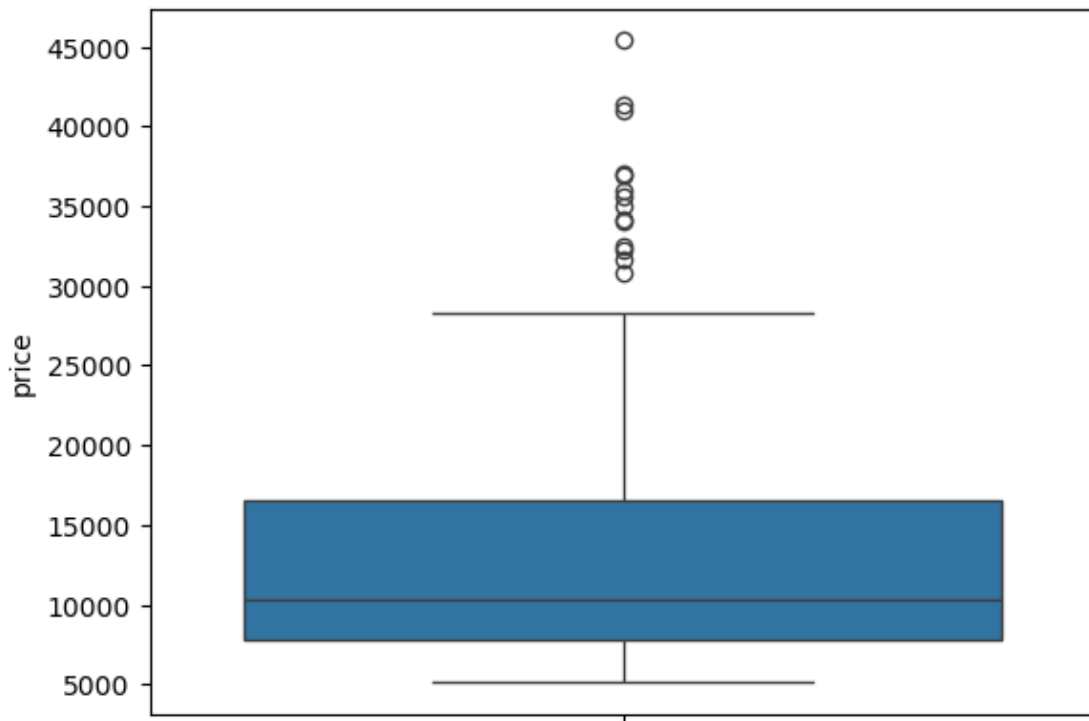
```
[170]:
```

	symboling	normalized-losses	width	height	engine-size \
count	205.000000	205.000000	205.000000	205.000000	205.000000
mean	0.834146	122.000000	65.907805	53.724878	126.907317
std	1.245307	31.681008	2.145204	2.443522	41.642693
min	-2.000000	65.000000	60.300000	47.800000	61.000000
25%	0.000000	101.000000	64.100000	52.000000	97.000000
50%	1.000000	122.000000	65.500000	54.100000	120.000000
75%	2.000000	137.000000	66.900000	55.500000	141.000000
max	3.000000	256.000000	72.300000	59.800000	326.000000

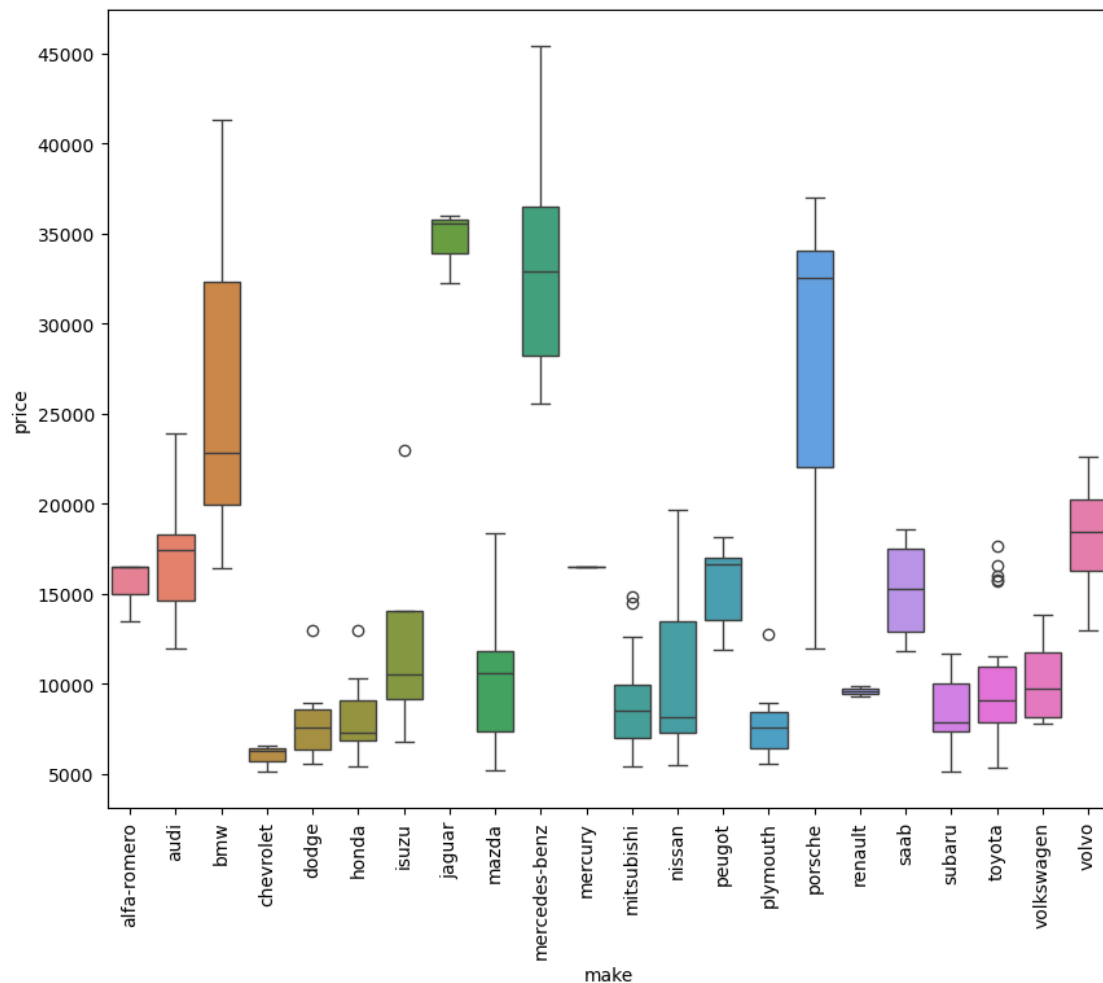
	horsepower	city-mpg	highway-mpg	price
count	205.000000	205.000000	205.000000	205.000000
mean	104.256158	25.219512	30.751220	13227.478049
std	39.519211	6.542142	6.886443	7902.651615
min	48.000000	13.000000	16.000000	5118.000000
25%	70.000000	19.000000	25.000000	7788.000000
50%	95.000000	24.000000	30.000000	10345.000000
75%	116.000000	30.000000	34.000000	16500.000000
max	288.000000	49.000000	54.000000	45400.000000

```
[171]: sns.boxplot(df['price'])
```

```
[171]: <Axes: ylabel='price'>
```



```
[172]: plt.figure(figsize=(10,8))
sns.boxplot(data=df,x='make',y='price',hue='make')
plt.xticks(rotation=90)
plt.show()
```



8 Dropping the outliers

```
[173]: df[(df['make']=='isuzu') & (df['price']>15000)]
```

```
[173]:   symboling  normalized-losses  make fuel-type body-style drive-wheels \
45         0             122.0  isuzu      gas      sedan           fwd

   engine-location  width  height engine-type  engine-size  horsepower \
45             front   63.6   52.0         ohc           90         70.0
```

	city-mpg	highway-mpg	price
45	38	43	23000

```
[174]: df.drop(45,inplace=True)
```

```
[175]: df[(df['make']=='dodge')&(df['price']>10000)]
```

```
[175]:      symboling  normalized-losses  make fuel-type body-style drive-wheels \
29          3          145.0  dodge      gas  hatchback          fwd

      engine-location  width  height engine-type  engine-size  horsepower \
29          front    66.3    50.2         ohc          156        145.0

      city-mpg  highway-mpg  price
29          19          24  12964
```

```
[176]: df.drop(29,inplace=True)
```

```
[177]: for make in df['make'].unique():
        print(make)
```

```
alfa-romero
audi
bmw
chevrolet
dodge
honda
isuzu
jaguar
mazda
mercedes-benz
mercury
mitsubishi
nissan
peugot
plymouth
porsche
renault
saab
subaru
toyota
volkswagen
volvo
```

```
[178]: for make in df['make'].unique():
        price=df[df['make']==make]['price']
        print(make,price)
```

```

alfa-romero 0      13495
1      16500
2      16500
Name: price, dtype: int64
audi 3      13950
4      17450
5      15250
6      17710
7      18920
8      23875
9      12000
Name: price, dtype: int64
bmw 10      16430
11      16925
12      20970
13      21105
14      24565
15      30760
16      41315
17      36880
Name: price, dtype: int64
chevrolet 18      5151
19      6295
20      6575
Name: price, dtype: int64
dodge 21      5572
22      6377
23      7957
24      6229
25      6692
26      7609
27      8558
28      8921
Name: price, dtype: int64
honda 30      6479
31      6855
32      5399
33      6529
34      7129
35      7295
36      7295
37      7895
38      9095
39      8845
40      10295
41      12945
42      10345
Name: price, dtype: int64

```

```

isuzu 43      6785
44      10000
46      11048
Name: price, dtype: int64
jaguar 47     32250
48      35550
49      36000
Name: price, dtype: int64
mazda 50      5195
51      6095
52      6795
53      6695
54      7395
55     10945
56     11845
57     13645
58     15645
59      8845
60      8495
61     10595
62     10245
63     10795
64     11245
65     18280
66     18344
Name: price, dtype: int64
mercedes-benz 67    25552
68     28248
69     28176
70     31600
71     34184
72     35056
73     40960
74     45400
Name: price, dtype: int64
mercury 75     16503
Name: price, dtype: int64
mitsubishi 76     5389
77      6189
78      6669
79      7689
80      9959
81      8499
82     12629
83     14869
84     14489
85      6989
86      8189

```

```

87      9279
88      9279
Name: price, dtype: int64
nissan 89      5499
90      7099
91      6649
92      6849
93      7349
94      7299
95      7799
96      7499
97      7999
98      8249
99      8949
100     9549
101    13499
102    14399
103    13499
104    17199
105    19699
106    18399
Name: price, dtype: int64
peugot 107     11900
108     13200
109     12440
110     13860
111     15580
112     16900
113     16695
114     17075
115     16630
116     17950
117     18150
Name: price, dtype: int64
plymouth 118     5572
119      7957
120      6229
121      6692
122      7609
123      8921
124     12764
Name: price, dtype: int64
porsche 125     22018
126     32528
127     34028
128     37028
129     12000
Name: price, dtype: int64

```

```

renault 130      9295
131      9895
Name: price, dtype: int64
saab 132      11850
133      12170
134      15040
135      15510
136      18150
137      18620
Name: price, dtype: int64
subaru 138      5118
139      7053
140      7603
141      7126
142      7775
143      9960
144      9233
145      11259
146      7463
147      10198
148      8013
149      11694
Name: price, dtype: int64
toyota 150      5348
151      6338
152      6488
153      6918
154      7898
155      8778
156      6938
157      7198
158      7898
159      7788
160      7738
161      8358
162      9258
163      8058
164      8238
165      9298
166      9538
167      8449
168      9639
169      9989
170      11199
171      11549
172      17669
173      8948
174      10698

```

```

175      9988
176     10898
177     11248
178     16558
179     15998
180     15690
181     15750
Name: price, dtype: int64
volkswagen 182      7775
183      7975
184      7995
185      8195
186      8495
187      9495
188      9995
189     11595
190      9980
191     13295
192     13845
193     12290
Name: price, dtype: int64
volvo 194     12940
195     13415
196     15985
197     16515
198     18420
199     18950
200     16845
201     19045
202     21485
203     22470
204     22625
Name: price, dtype: int64

```

```

[179]: for make in df['make'].unique():
        price=df[df['make']==make]['price']
        q1=np.quantile(price,0.25)
        q3=np.quantile(price,0.75)
        iqr=q3-q1
        uw=q3+(1.5*iqr)
        lw=q1-(1.5*iqr)
        print(make,iqr)
        print(make,uw)
        print(make,lw)

```

```

alfa-romero 1502.5
alfa-romero 18753.75
alfa-romero 12743.75

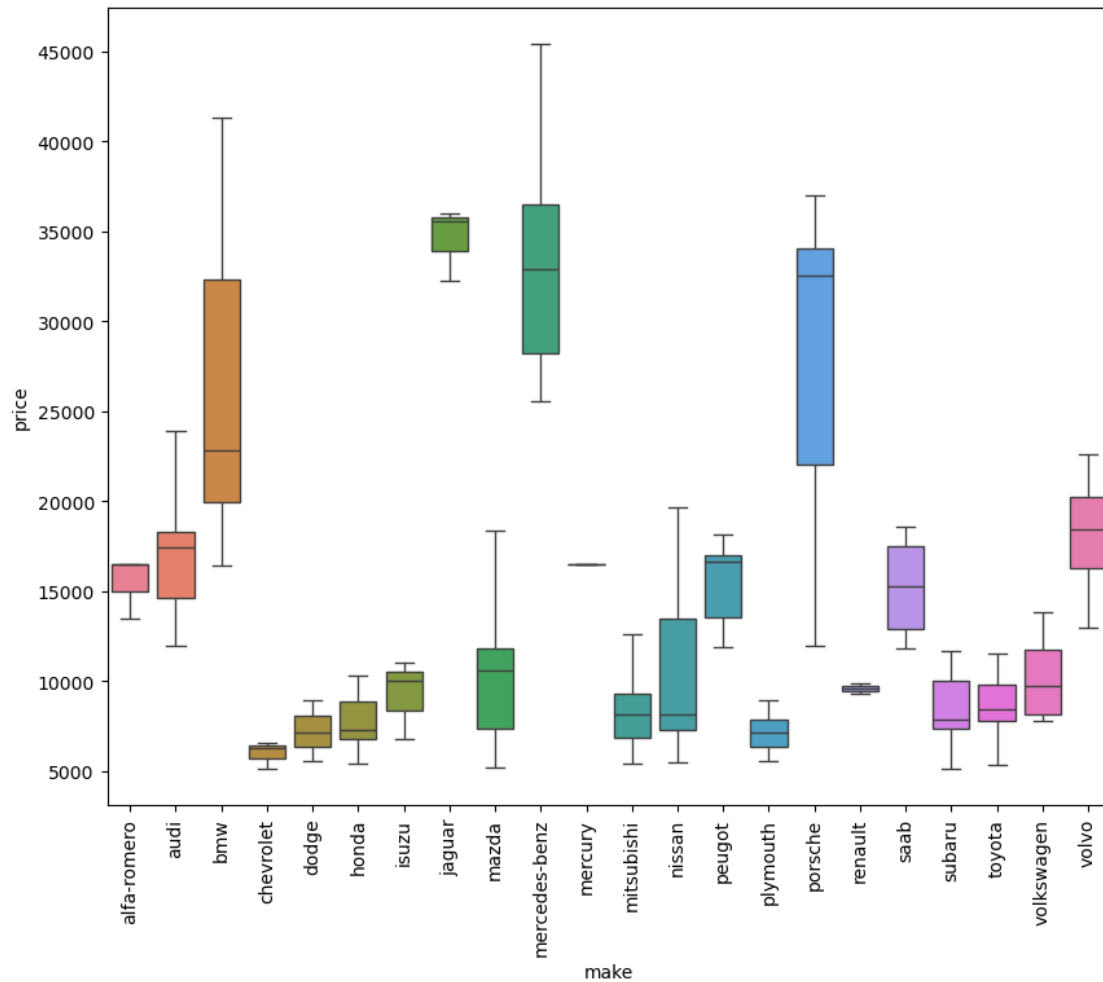
```

audi 3715.0
audi 23887.5
audi 9027.5
bmw 12331.25
bmw 50786.875
bmw 1461.875
chevrolet 712.0
chevrolet 7503.0
chevrolet 4655.0
dodge 1767.25
dodge 10758.125
dodge 3689.125
honda 2240.0
honda 12455.0
honda 3495.0
isuzu 2131.5
isuzu 13721.25
isuzu 5195.25
jaguar 1875.0
jaguar 38587.5
jaguar 31087.5
mazda 4450.0
mazda 18520.0
mazda 720.0
mercedes-benz 8302.0
mercedes-benz 48985.0
mercedes-benz 15777.0
mercury 0.0
mercury 16503.0
mercury 16503.0
mitsubishi 2970.0
mitsubishi 14414.0
mitsubishi 2534.0
nissan 6187.5
nissan 22780.25
nissan -1969.75
peugot 3457.5
peugot 22173.75
peugot 8343.75
plymouth 1978.5
plymouth 11406.75
plymouth 3492.75
porsche 12010.0
porsche 52043.0
porsche 4003.0
renault 300.0
renault 10195.0
renault 8995.0

```
saab 4602.5
saab 24393.75
saab 5983.75
subaru 2640.75
subaru 13980.625
subaru 3417.625
toyota 3102.75
toyota 15627.375
toyota 3216.375
volkswagen 3623.75
volkswagen 17204.375
volkswagen 2709.375
volvo 4015.0
volvo 26287.5
volvo 10227.5
```

```
[180]: for make in df['make'].unique():
        price=df[df['make']==make]['price']
        q1=np.quantile(price,0.25)
        q3=np.quantile(price,0.75)
        iqr=q3-q1
        uw=q3+(1.5*iqr)
        lw=q1-(1.5*iqr)
        index_above_uw=price[price>uw].index
        index_below_lw=price[price<lw].index
        #print(index_above_uw)
        #print(index_below_lw)
        if len(index_above_uw) !=0:
            df.drop(index_above_uw,inplace=True)
        if len(index_below_lw) !=0:
            df.drop(index_below_lw,inplace=True)
```

```
[181]: plt.figure(figsize=(10,8))
        sns.boxplot(data=df,x='make',y='price',hue='make')
        plt.xticks(rotation=90)
        plt.show()
```



9 Split the Dataframe into numerical & categorical Dataframes

```
[182]: df_cat=df.select_dtypes('object')
df_cat
```

```
[182]:
```

	make	fuel-type	body-style	drive-wheels	engine-location	\
0	alfa-romero	gas	convertible	rwd	front	
1	alfa-romero	gas	convertible	rwd	front	
2	alfa-romero	gas	hatchback	rwd	front	
3	audi	gas	sedan	fwd	front	
4	audi	gas	sedan	4wd	front	
..	...	...	...	...	...	
200	volvo	gas	sedan	rwd	front	
201	volvo	gas	sedan	rwd	front	
202	volvo	gas	sedan	rwd	front	

203	volvo	diesel	sedan	rwd	front
204	volvo	gas	sedan	rwd	front

```

engine-type
0      dohc
1      dohc
2      ohcv
3      ohc
4      ohc
..      ...
200     ohc
201     ohc
202     ohcv
203     ohc
204     ohc

```

[194 rows x 6 columns]

```
[183]: df_num=df.select_dtypes(['float64','int64'])
df_num
```

```
[183]:
```

	symboling	normalized-losses	width	height	engine-size	horsepower	\
0	3	122.0	64.1	48.8	130	111.0	
1	3	122.0	64.1	48.8	130	111.0	
2	1	122.0	65.5	52.4	152	154.0	
3	2	164.0	66.2	54.3	109	102.0	
4	2	164.0	66.4	54.3	136	115.0	
..	...	...	...	...	...	...	
200	-1	95.0	68.9	55.5	141	114.0	
201	-1	95.0	68.8	55.5	141	160.0	
202	-1	95.0	68.9	55.5	173	134.0	
203	-1	95.0	68.9	55.5	145	106.0	
204	-1	95.0	68.9	55.5	141	114.0	

	city-mpg	highway-mpg	price
0	21	27	13495
1	21	27	16500
2	19	26	16500
3	24	30	13950
4	18	22	17450
..	...	...	...
200	23	28	16845
201	19	25	19045
202	18	23	21485
203	26	27	22470
204	19	25	22625

[194 rows x 9 columns]

10 Handling Categorical Data

Most of the ML algorithm doesn't understand the values of type object. In encoding we will convert the categorical data into numerical form

- One Hot Encoding
- Label Encoding

One Hot Encoding

- This encoding will create a binary column for each unique category in the categorical feature where each column will contain 0s and 1s to indicate the presence of the category.
- It is preferable when the column contain few unique categories

```
[184]: df_cat['fuel-type'].value_counts()
```

```
[184]: fuel-type
gas      174
diesel   20
Name: count, dtype: int64
```

```
[185]: fuel_data=pd.get_dummies(df_cat['fuel-type'])
fuel_data
```

```
[185]:      diesel    gas
0      False    True
1      False    True
2      False    True
3      False    True
4      False    True
..      ...      ...
200     False    True
201     False    True
202     False    True
203      True   False
204     False    True
```

[194 rows x 2 columns]

```
[186]: fuel_data=pd.get_dummies(df_cat['fuel-type']).astype(int)
fuel_data
```

```
[186]:      diesel    gas
0         0     1
1         0     1
2         0     1
```

```

3         0     1
4         0     1
..      ...   ...
200        0     1
201        0     1
202        0     1
203        1     0
204        0     1

```

```
[194 rows x 2 columns]
```

11 Label Encoding

- Here each unique category in the categorical feature is assigned a sequential integer starting from 0

```
[187]: %pip install scikit-learn
```

```

Requirement already satisfied: scikit-learn in c:\anacondanew\lib\site-packages
(1.6.1)
Requirement already satisfied: numpy>=1.19.5 in c:\anacondanew\lib\site-packages
(from scikit-learn) (2.1.3)
Requirement already satisfied: scipy>=1.6.0 in c:\anacondanew\lib\site-packages
(from scikit-learn) (1.15.3)
Requirement already satisfied: joblib>=1.2.0 in c:\anacondanew\lib\site-packages
(from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\anacondanew\lib\site-
packages (from scikit-learn) (3.5.0)
Note: you may need to restart the kernel to use updated packages.

```

```
[188]: from sklearn.preprocessing import LabelEncoder
```

```
[189]: le=LabelEncoder() #creating object of the label encoder class
```

fit-transform method is used in many Machine Learning process to fit a transformation to the data in a single step:

- fit: learn what needs to be done from the data (like categories for encoding or mean, standard deviation, min, max for scaling)
- transform: applies the learned changes to the data

```
[190]: df_cat['make']=le.fit_transform(df_cat['make'])
```

```
[191]: df_cat['make']
```

```

[191]: 0         0
       1         0
       2         0
       3         1

```

```

4      1
      ..
200    21
201    21
202    21
203    21
204    21
Name: make, Length: 194, dtype: int64

```

```
[192]: df_cat['body-style']=le.fit_transform(df_cat['body-style'])
```

```
[193]: df_cat['body-style']
```

```

[193]: 0      0
      1      0
      2      2
      3      3
      4      3
      ..
200    3
201    3
202    3
203    3
204    3
Name: body-style, Length: 194, dtype: int64

```

```
[194]: for column in df_cat:
        print(column)
```

```

make
fuel-type
body-style
drive-wheels
engine-location
engine-type

```

```
[195]: for column in df_cat:
        le=LabelEncoder()
        df_cat[column]=le.fit_transform(df_cat[column])
```

```
[196]: df_cat
```

```

[196]:   make  fuel-type  body-style  drive-wheels  engine-location  engine-type
0      0          1           0             2             0           0
1      0          1           0             2             0           0
2      0          1           2             2             0           5
3      1          1           3             1             0           3
4      1          1           3             0             0           3

```

200	21	1	3	2	0	3
201	21	1	3	2	0	3
202	21	1	3	2	0	5
203	21	0	3	2	0	3
204	21	1	3	2	0	3

[194 rows x 6 columns]

12 Handling Skewness

13 The Data can be left or right skew

- left skewed $\rightarrow \text{mean} < \text{median} < \text{mode}$
- right skewed $\rightarrow \text{mean} > \text{median} > \text{mode}$

```
[197]: df_num.describe()
```

```
[197]:
```

	symboling	normalized-losses	width	height	engine-size \
count	194.000000	194.000000	194.000000	194.000000	194.000000
mean	0.788660	121.402062	65.892268	53.822165	125.628866
std	1.200896	31.406299	2.189327	2.448045	42.036826
min	-2.000000	65.000000	60.300000	47.800000	61.000000
25%	0.000000	101.000000	64.000000	52.000000	97.000000
50%	1.000000	122.000000	65.400000	54.100000	110.000000
75%	2.000000	137.000000	66.900000	55.675000	140.750000
max	3.000000	256.000000	72.300000	59.800000	326.000000

	horsepower	city-mpg	highway-mpg	price
count	194.000000	194.000000	194.000000	194.000000
mean	102.430476	25.412371	30.984536	13087.304124
std	39.307294	6.546371	6.882039	8075.197621
min	48.000000	13.000000	16.000000	5118.000000
25%	70.000000	19.250000	25.000000	7747.250000
50%	95.000000	25.000000	30.500000	9992.000000
75%	116.000000	30.000000	36.000000	16500.000000
max	288.000000	49.000000	54.000000	45400.000000

```
[198]: %pip install scipy
```

```
Requirement already satisfied: scipy in c:\anacondanew\lib\site-packages
(1.15.3)
Requirement already satisfied: numpy<2.5,>=1.23.5 in c:\anacondanew\lib\site-
packages (from scipy) (2.1.3)
Note: you may need to restart the kernel to use updated packages.
```

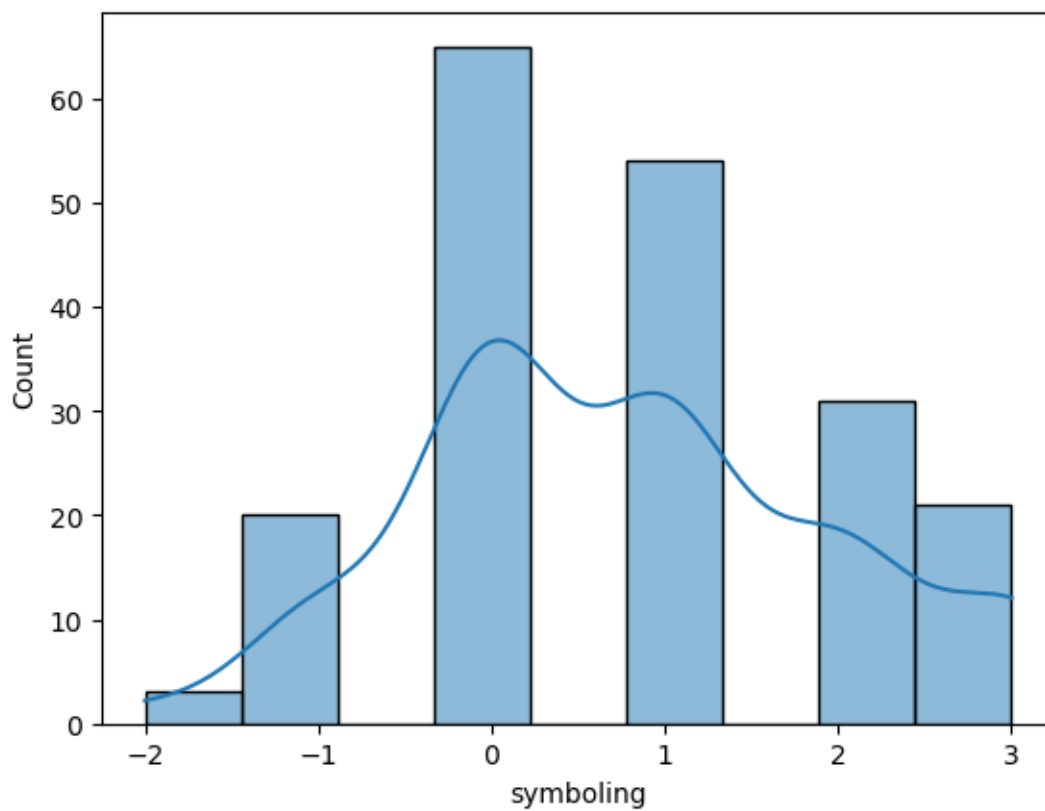
```
[199]: from scipy.stats import skew
```

```
[200]: for col in df_num:
        print(col)
        print(skew(df_num[col]))
        print('-----')
```

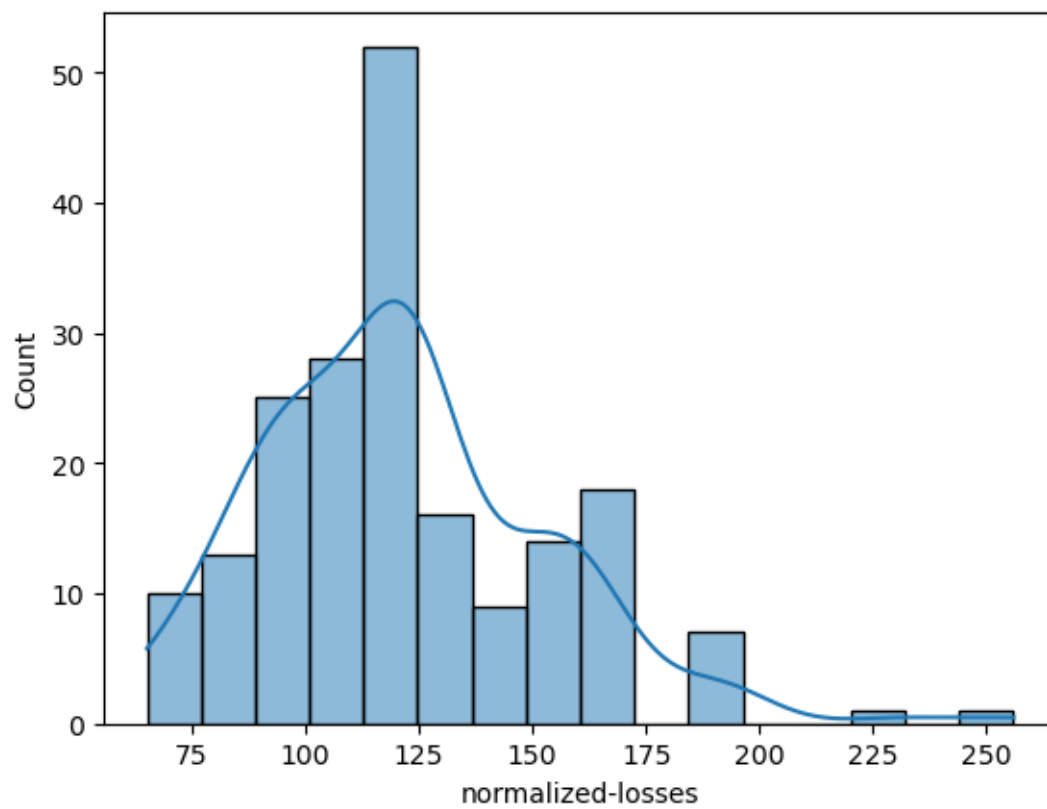
```
symboling
0.21386866184357742
-----
normalized-losses
0.848205953606264
-----
width
0.9140400320504322
-----
height
0.013839962443639326
-----
engine-size
2.0541257626466156
-----
horsepower
1.5556576549504106
-----
city-mpg
0.5999073033714895
-----
highway-mpg
0.4760310091695327
-----
price
1.8166794702973463
-----
```

```
[201]: for col in df_num:
        print(col)
        print(skew(df_num[col]))
        plt.figure()
        sns.histplot(df_num[col],kde=True)
        plt.show()
```

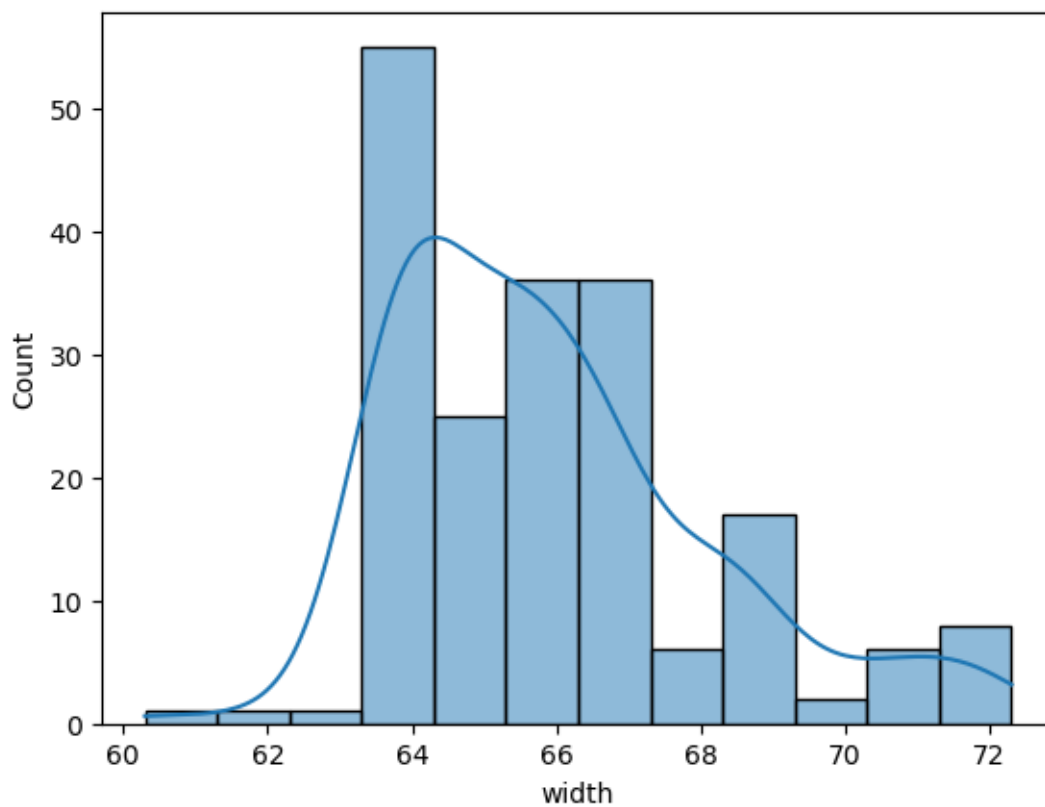
```
symboling
0.21386866184357742
```



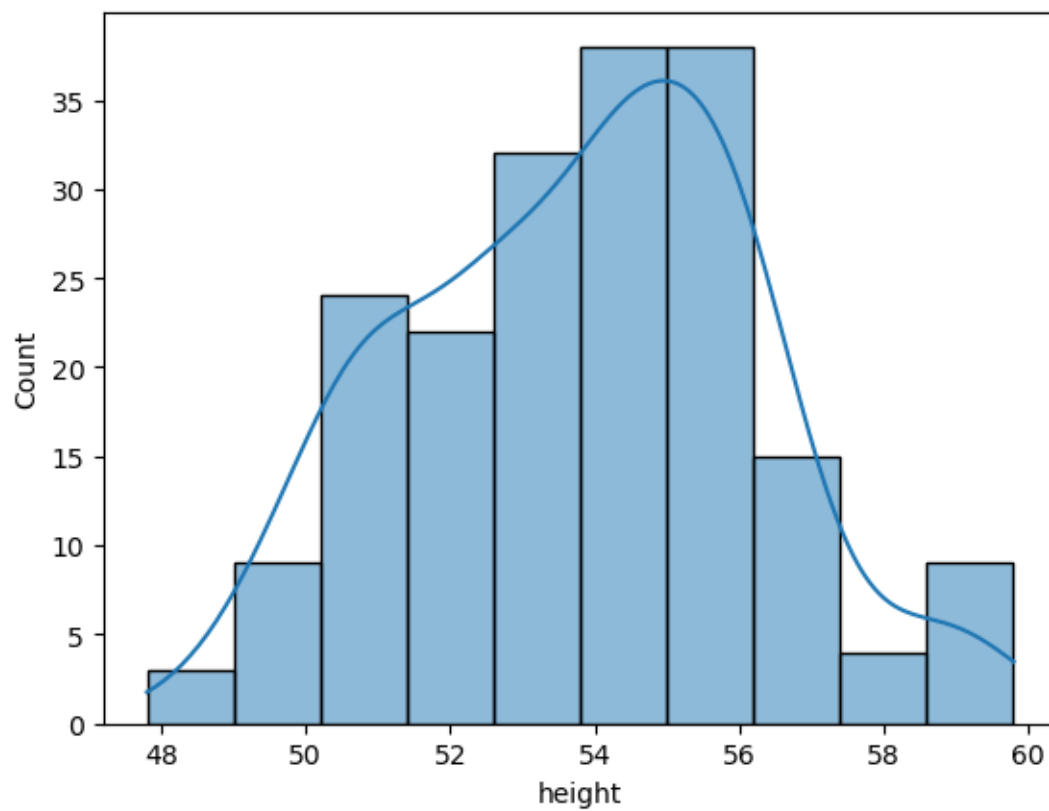
normalized-losses
0.848205953606264



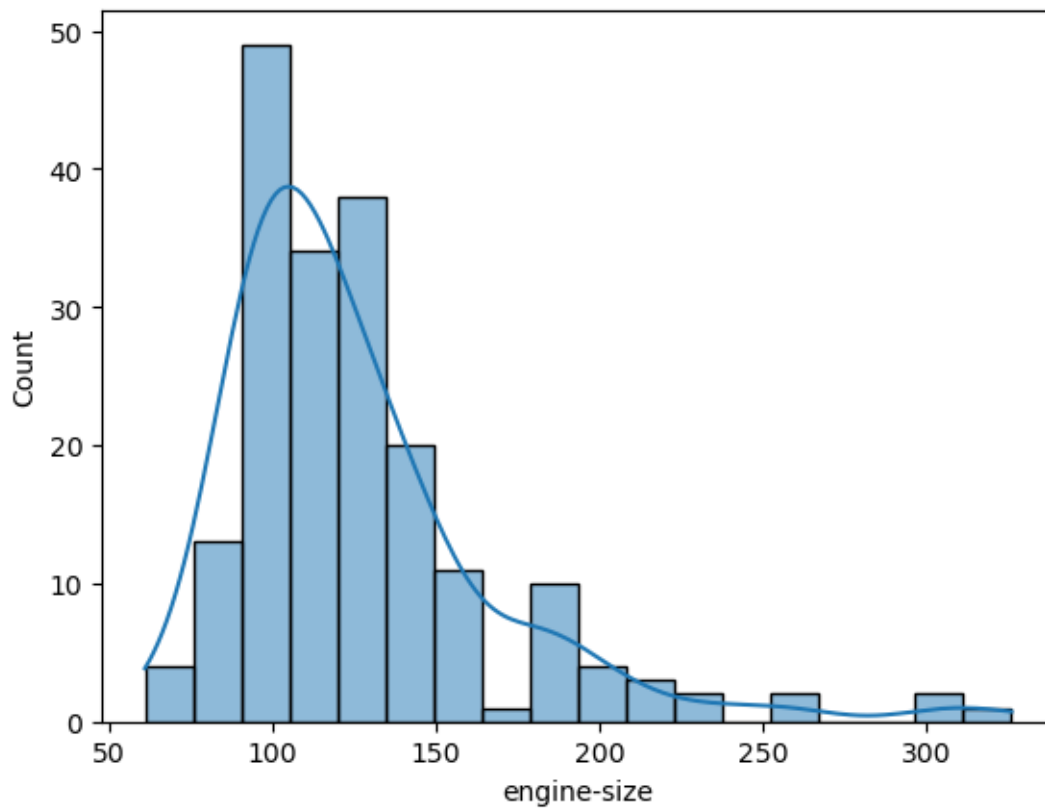
width
0.9140400320504322



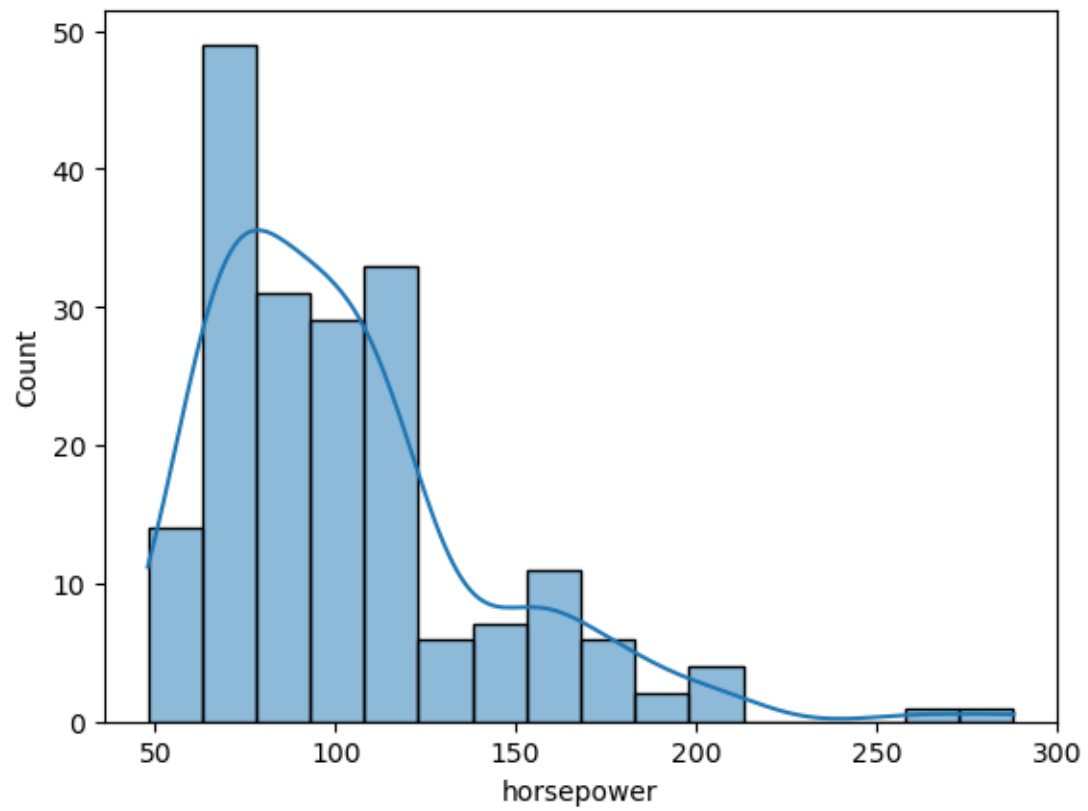
height
0.013839962443639326



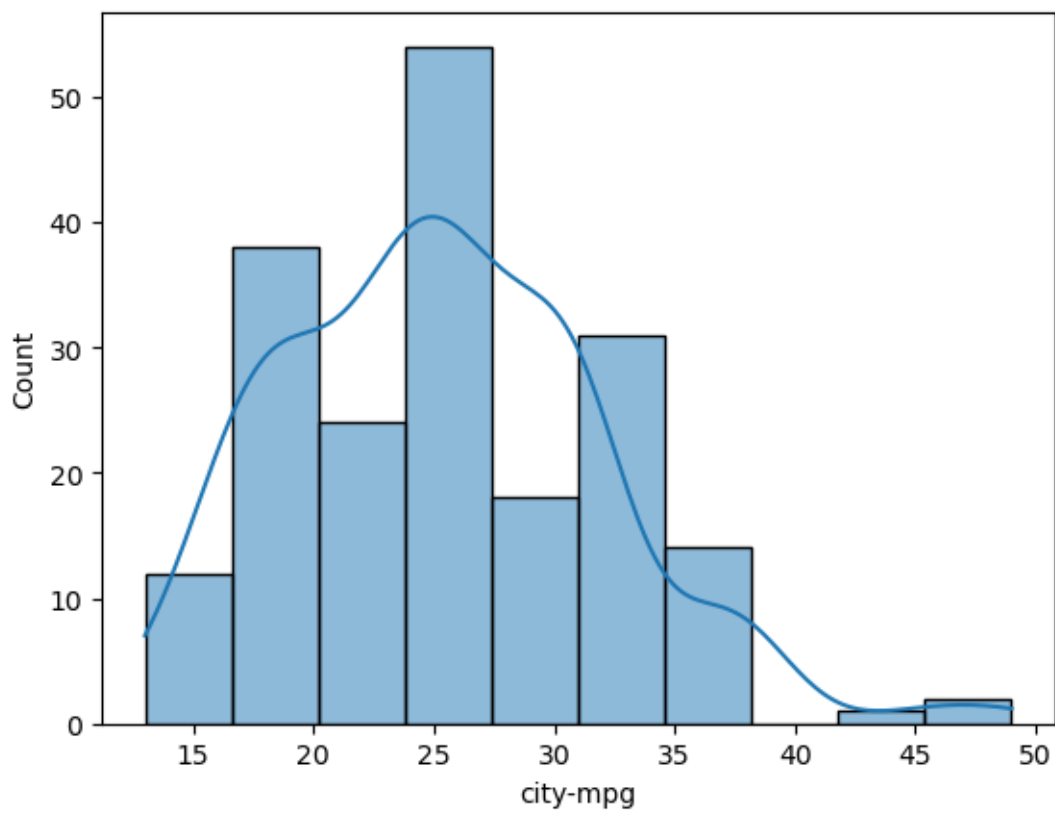
engine-size
2.0541257626466156



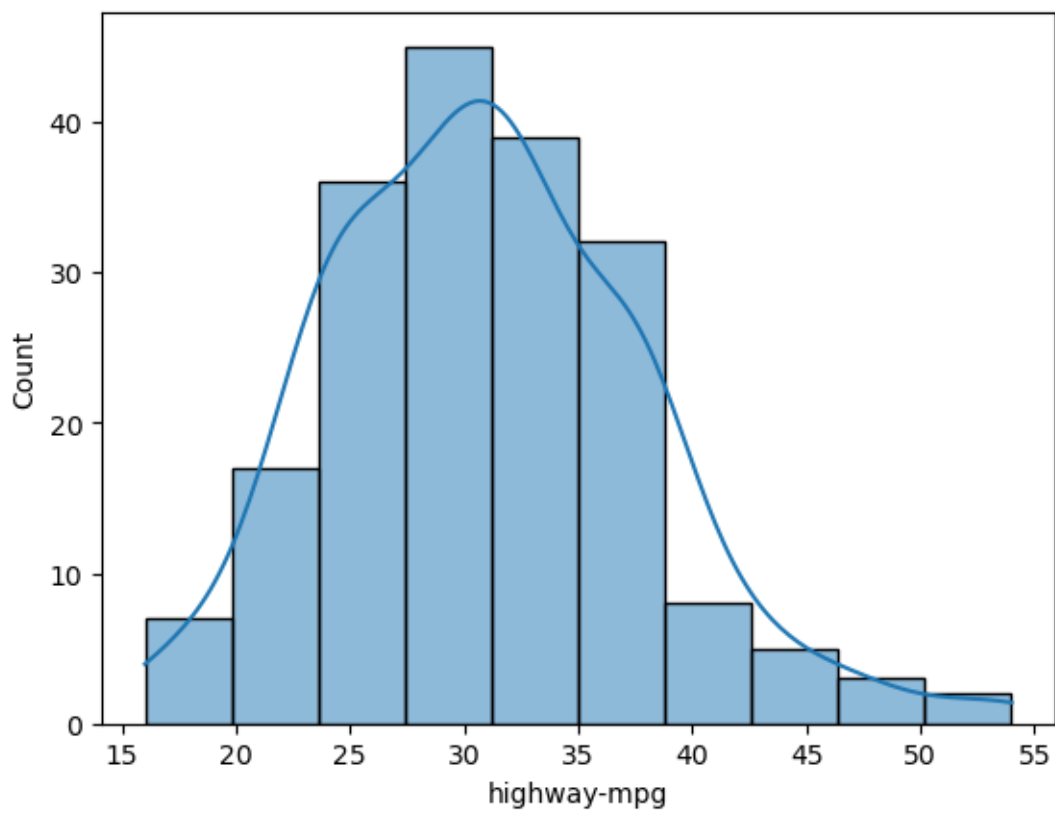
horsepower
1.5556576549504106



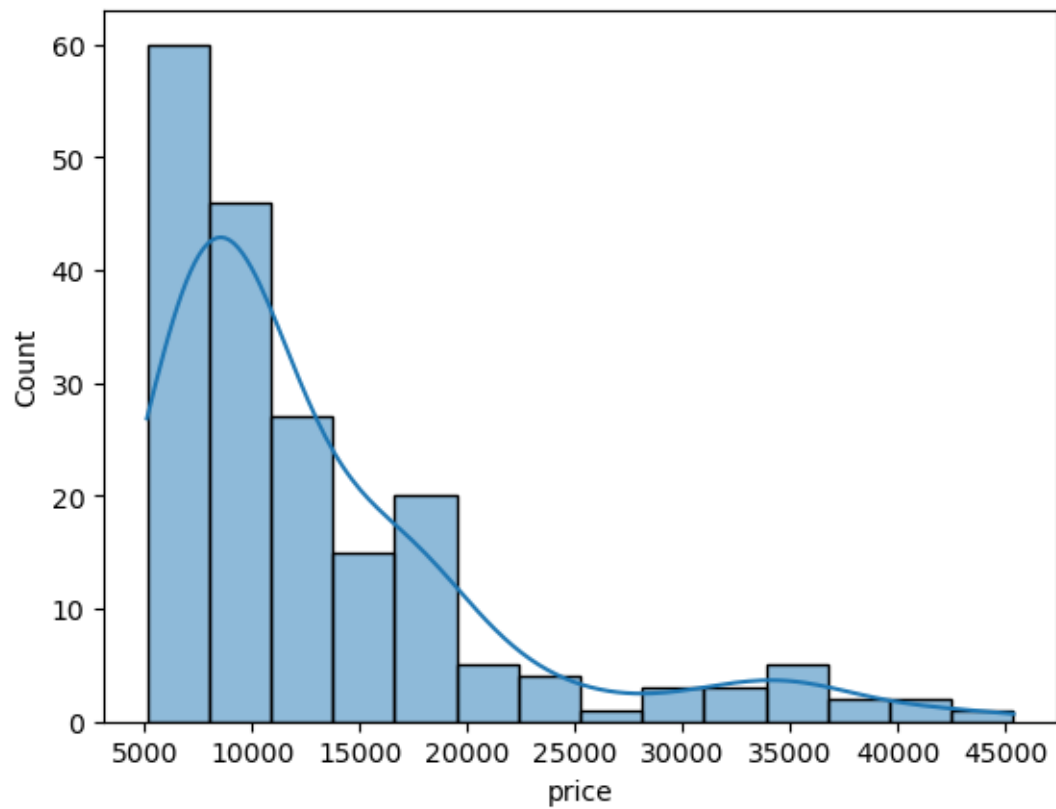
city-mpg
0.5999073033714895



highway-mpg
0.4760310091695327

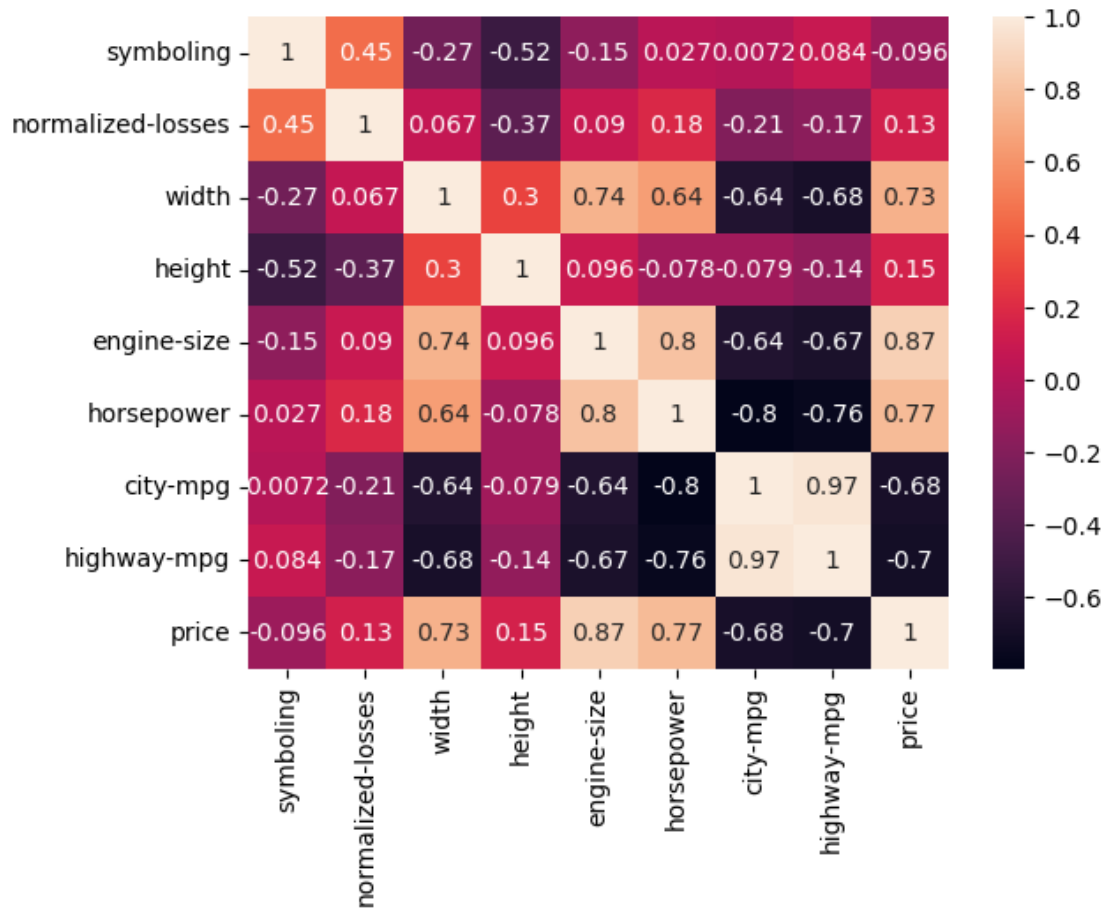


price
1.8166794702973463



```
[202]: sns.heatmap(df_num.corr(),annot=True)
```

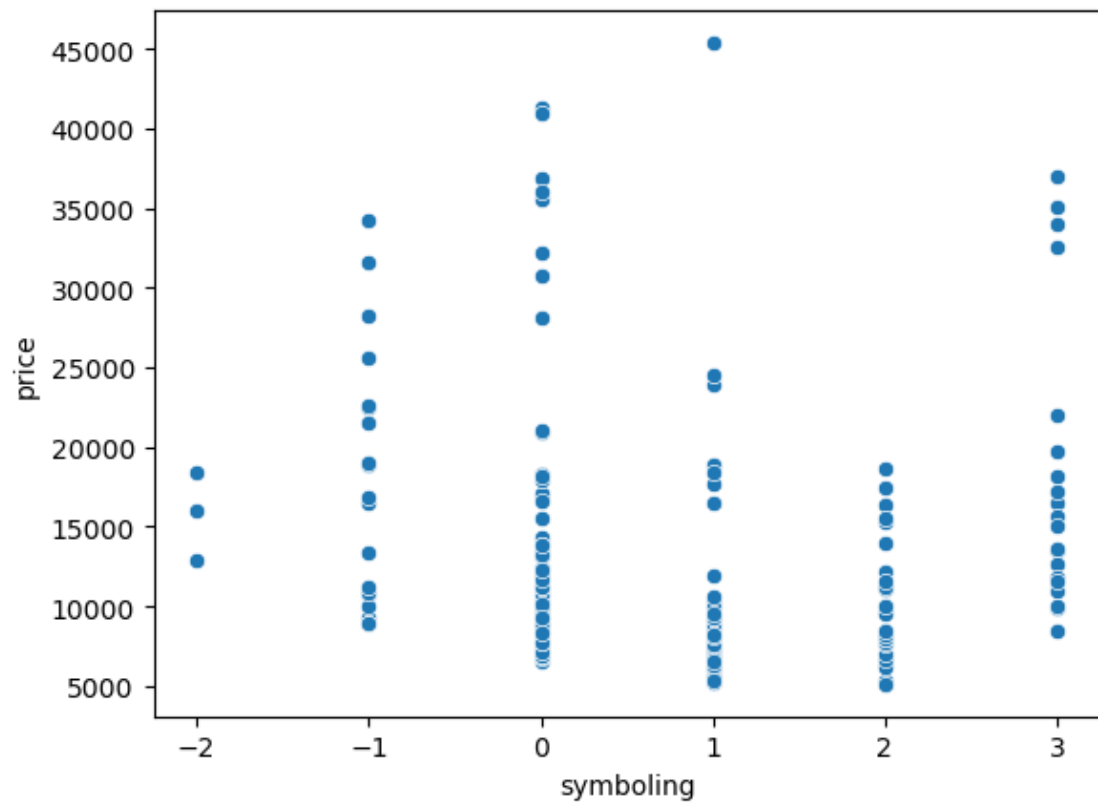
```
[202]: <Axes: >
```



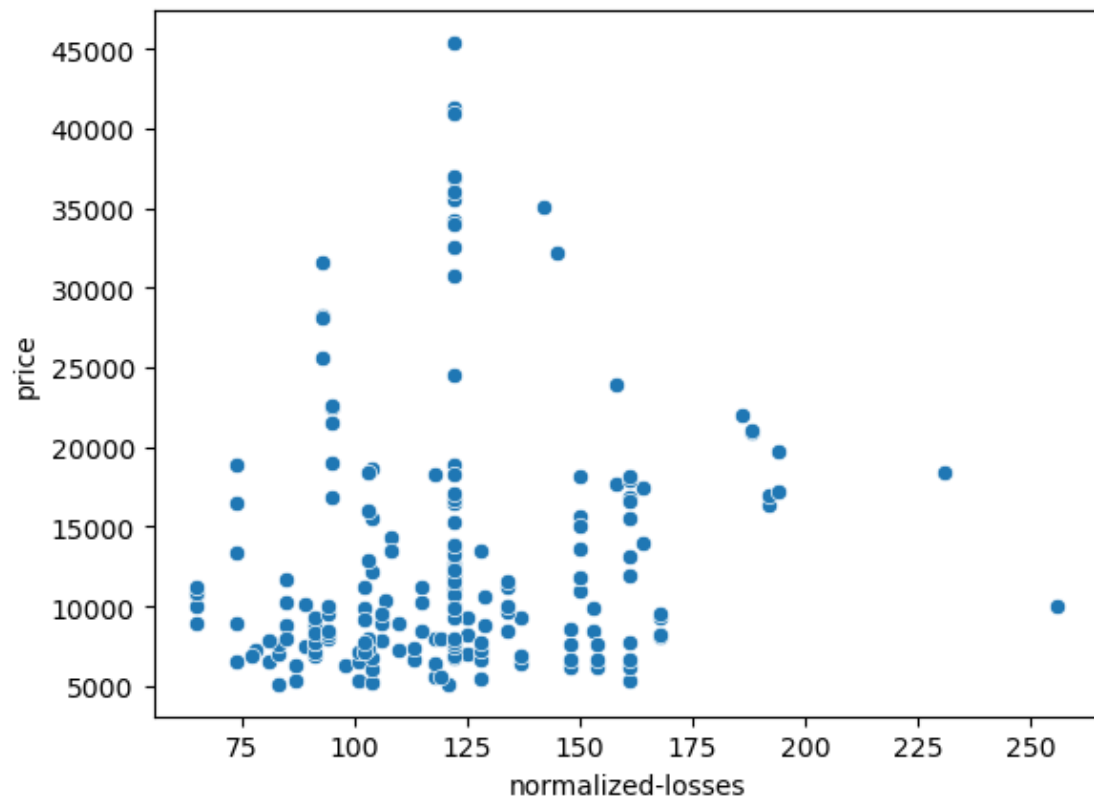
```
[203]: for col in df_num:
        print(col)

        plt.figure()
        sns.scatterplot(data=df_num,y='price',x=col)
        plt.show()
```

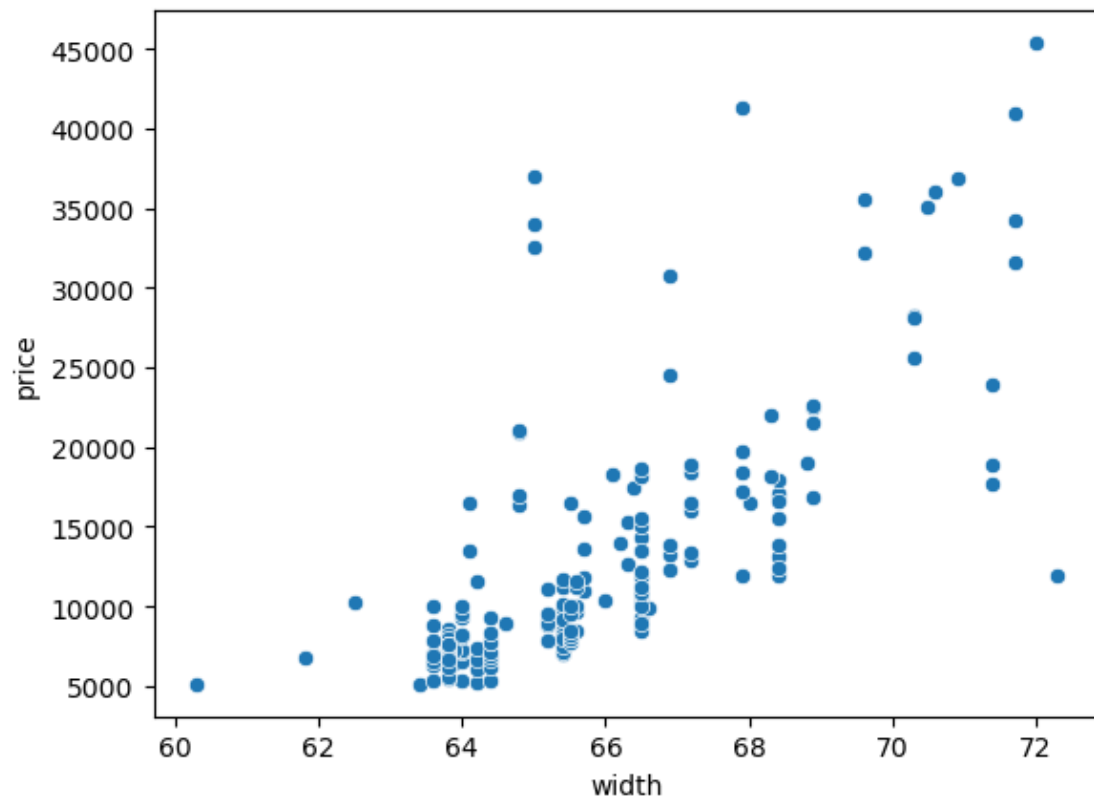
symboling



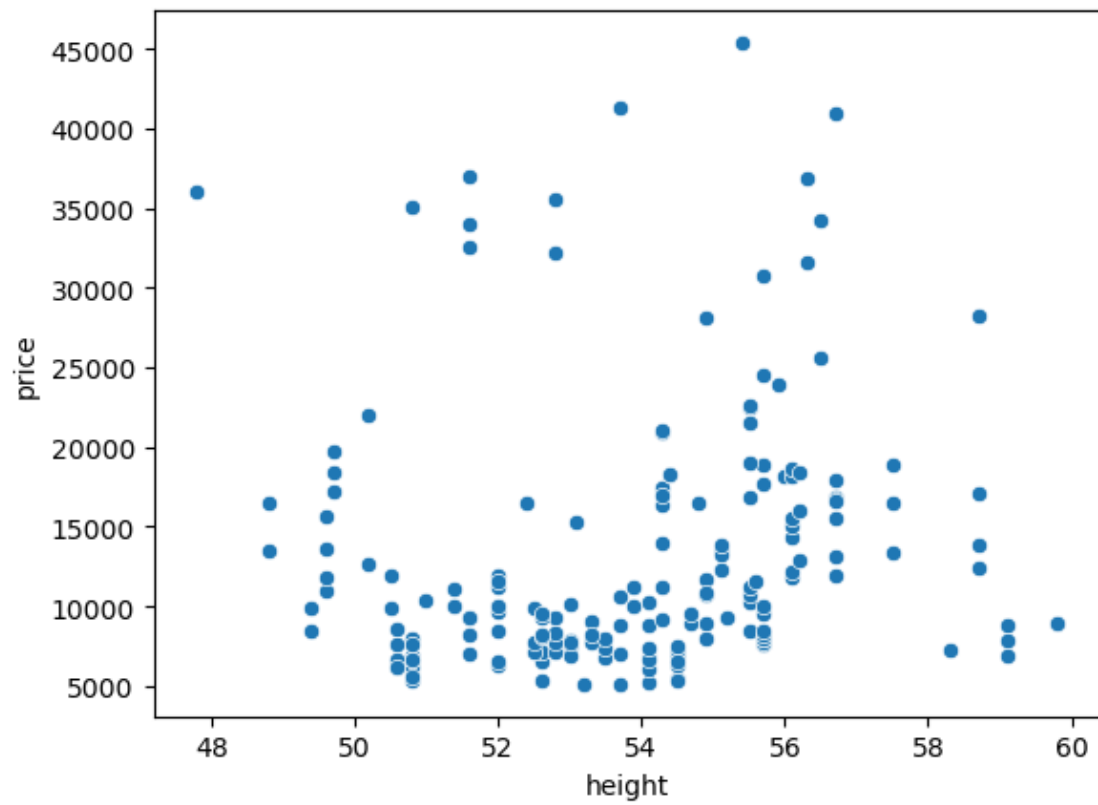
normalized-losses



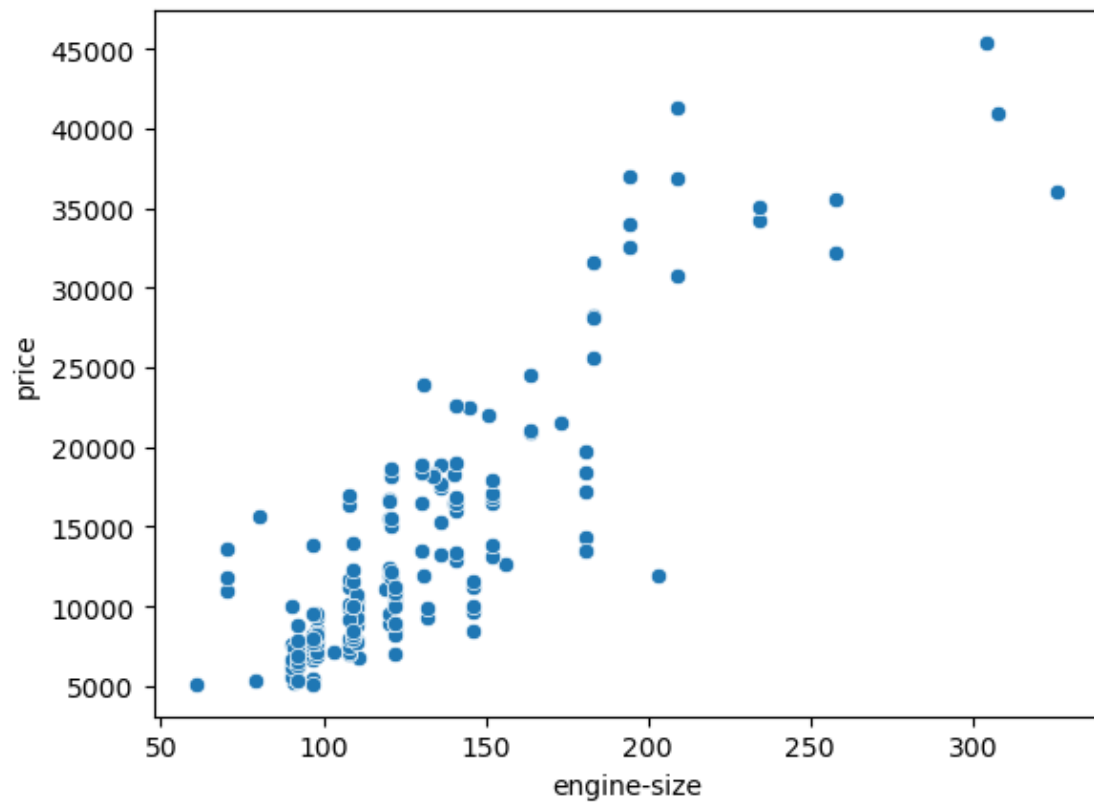
width



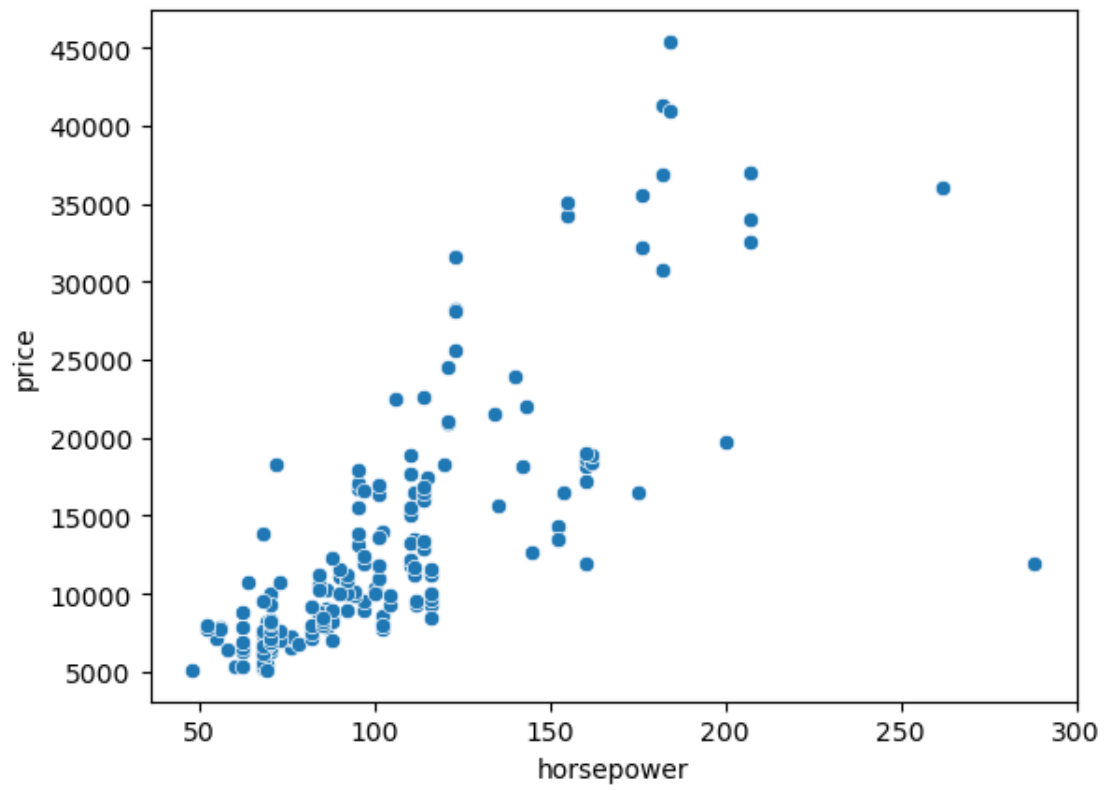
height



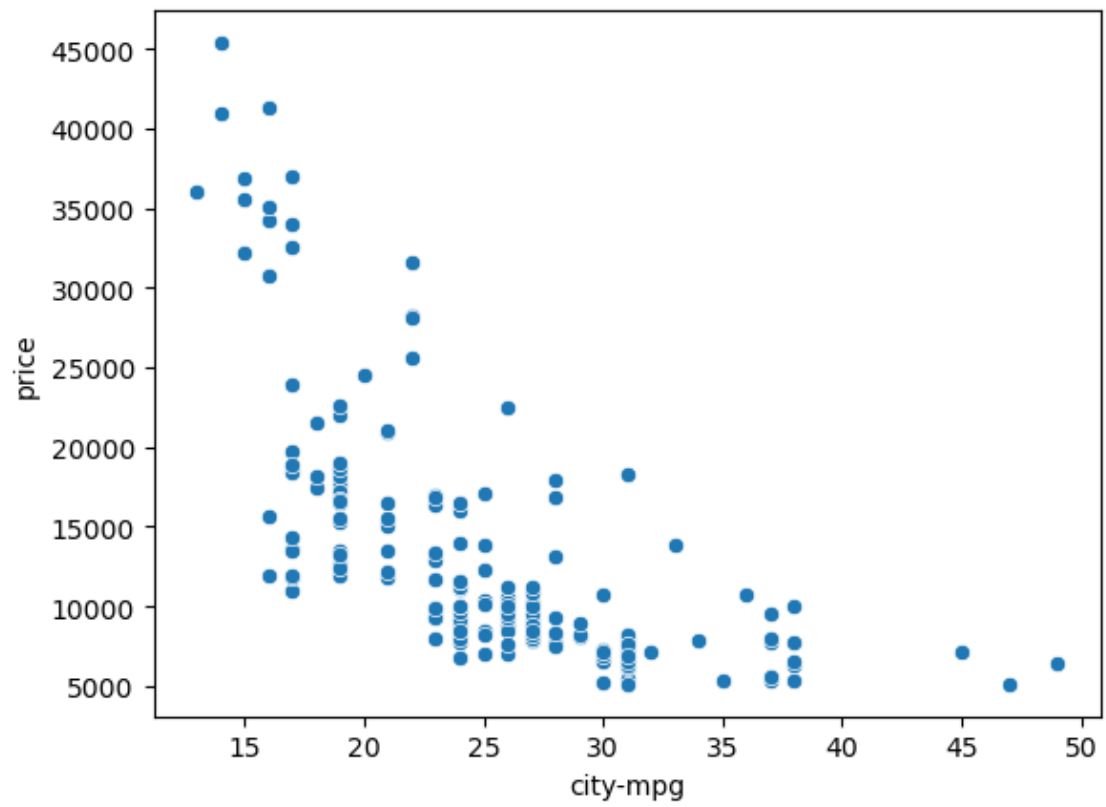
engine-size



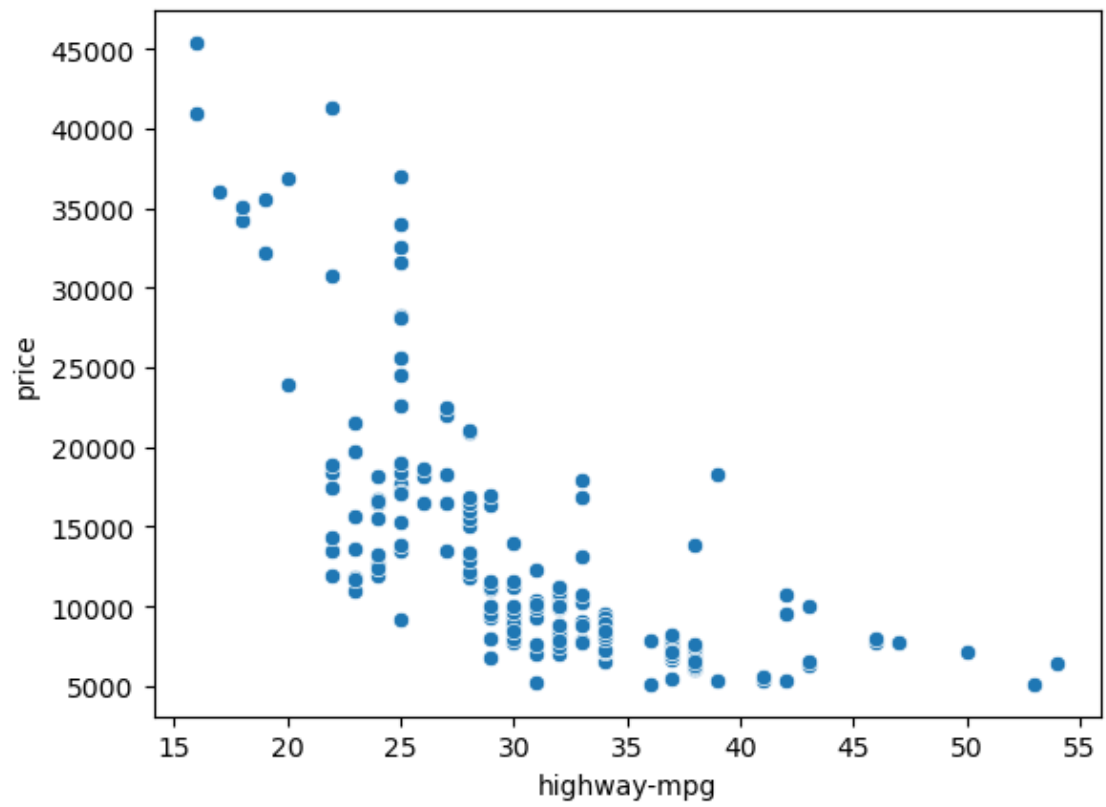
horsepower



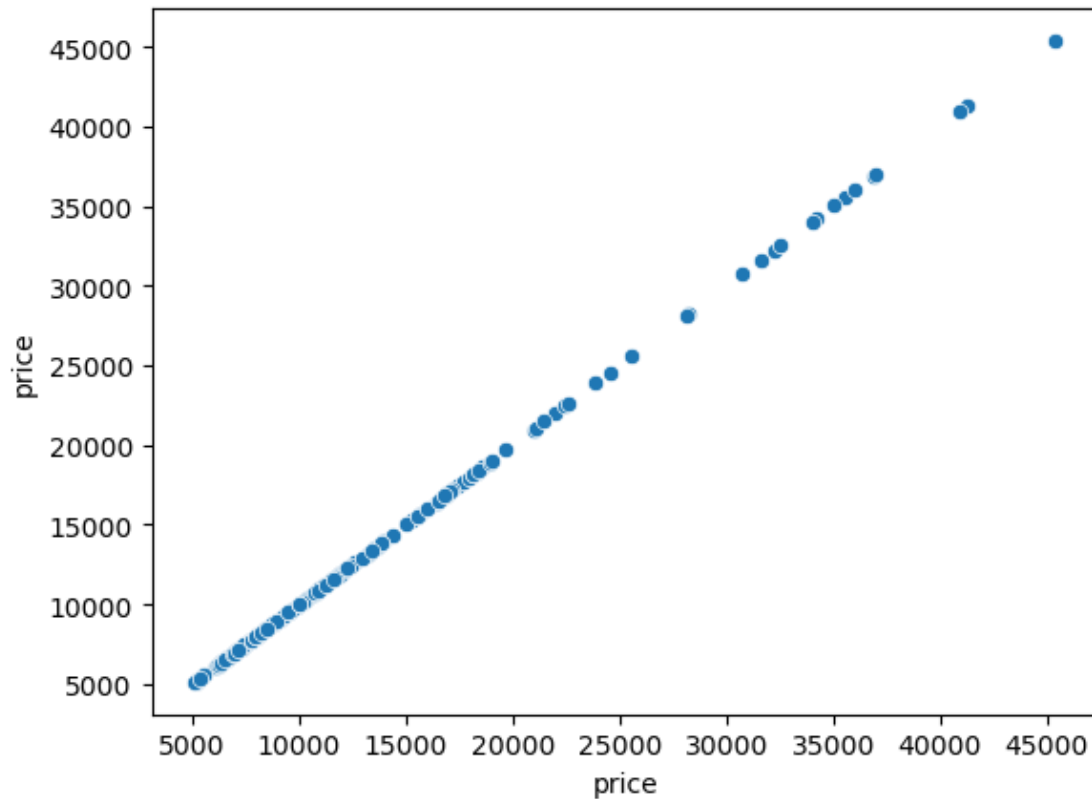
city-mpg



highway-mpg



price



Reduce the skewness from the data

- applying the square root of the particular
- applying the log transformation on the particular column

Square Root

- Here the impact of the larger values is noticed as compared to smaller values

Log Transformation

- Significantly reduce the impact of the larger values

```
[204]: skew(df_num['normalized-losses'])
```

```
[204]: np.float64(0.848205953606264)
```

```
[205]: #df_num['normalized-losses']=np.sqrt(df['normalized-losses'])
```

```
[206]: #skew(df_num['normalized-losses'])
```

```
[207]: df_num['normalized-losses']=np.log(df['normalized-losses'])
```

```
[208]: skew(df_num['normalized-losses'])
```

```
[208]: np.float64(0.018468899462087518)
```

```
[209]: np.sqrt(-2)
```

```
[209]: np.float64(nan)
```

```
[210]: np.log(-5)
```

```
[210]: np.float64(nan)
```

```
[211]: df_num.head()
```

```
[211]:
```

	symboling	normalized-losses	width	height	engine-size	horsepower	\
0	3	4.804021	64.1	48.8	130	111.0	
1	3	4.804021	64.1	48.8	130	111.0	
2	1	4.804021	65.5	52.4	152	154.0	
3	2	5.099866	66.2	54.3	109	102.0	
4	2	5.099866	66.4	54.3	136	115.0	

	city-mpg	highway-mpg	price
0	21	27	13495
1	21	27	16500
2	19	26	16500
3	24	30	13950
4	18	22	17450

Scaling Scaling is the process to transform the data for the different unit or different features on the same scale, making it easy to compare.

- Min-Max Scalar (Normalization)
- Standard Scalar (Standardization)

Min-Max Scalar

$$13.0.1 \quad x_{\text{new}} = \frac{x_{\text{old}} - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

```
[212]: from sklearn.preprocessing import MinMaxScaler
```

```
[213]: mm=MinMaxScaler()
```

```
[214]: #df_num['normalized-losses']=mm.fit_transform(df_num[['normalized-losses']])
```

```
[215]: df_num
```

```
[215]:
```

	symboling	normalized-losses	width	height	engine-size	horsepower	\
0	3	4.804021	64.1	48.8	130	111.0	
1	3	4.804021	64.1	48.8	130	111.0	
2	1	4.804021	65.5	52.4	152	154.0	

3	2	5.099866	66.2	54.3	109	102.0
4	2	5.099866	66.4	54.3	136	115.0
..	...	...	...	...	...	...
200	-1	4.553877	68.9	55.5	141	114.0
201	-1	4.553877	68.8	55.5	141	160.0
202	-1	4.553877	68.9	55.5	173	134.0
203	-1	4.553877	68.9	55.5	145	106.0
204	-1	4.553877	68.9	55.5	141	114.0

	city-mpg	highway-mpg	price
0	21	27	13495
1	21	27	16500
2	19	26	16500
3	24	30	13950
4	18	22	17450
..	...	...	...
200	23	28	16845
201	19	25	19045
202	18	23	21485
203	26	27	22470
204	19	25	22625

[194 rows x 9 columns]

Standard Scalar

13.0.2 $z = \frac{x - \text{mean}}{\text{standard deviation}}$

```
[216]: from sklearn.preprocessing import StandardScaler
```

```
[217]: ss=StandardScaler()
```

```
[218]: df_num['city-mpg']=ss.fit_transform(df_num[['city-mpg']])
```

```
[219]: df_num
```

```
[219]:
```

	symboling	normalized-losses	width	height	engine-size	horsepower	\
0	3	4.804021	64.1	48.8	130	111.0	
1	3	4.804021	64.1	48.8	130	111.0	
2	1	4.804021	65.5	52.4	152	154.0	
3	2	5.099866	66.2	54.3	109	102.0	
4	2	5.099866	66.4	54.3	136	115.0	
..	...	...	...	...	...	...	
200	-1	4.553877	68.9	55.5	141	114.0	
201	-1	4.553877	68.8	55.5	141	160.0	
202	-1	4.553877	68.9	55.5	173	134.0	
203	-1	4.553877	68.9	55.5	145	106.0	

204	-1	4.553877	68.9	55.5	141	114.0
-----	----	----------	------	------	-----	-------

	city-mpg	highway-mpg	price
0	-0.675762	27	13495
1	-0.675762	27	16500
2	-0.982065	26	16500
3	-0.216307	30	13950
4	-1.135217	22	17450
..	...	...	...
200	-0.369459	28	16845
201	-0.982065	25	19045
202	-1.135217	23	21485
203	0.089996	27	22470
204	-0.982065	25	22625

[194 rows x 9 columns]

```
[220]: for col in df_num:
        ss=StandardScaler()
        df_num[col]=ss.fit_transform(df_num[[col]])
```

```
[221]: df_num
```

```
[221]:      symboling  normalized-losses      width      height  engine-size  \
0      1.846173      0.146152 -0.820757 -2.056808      0.104252
1      1.846173      0.146152 -0.820757 -2.056808      0.104252
2      0.176441      0.146152 -0.179636 -0.582442      0.628957
3      1.011307      1.315366  0.140924  0.195695     -0.396602
4      1.011307      1.315366  0.232512  0.195695      0.247354
..      ...      ...      ...      ...      ...
200    -1.493292     -0.842445  1.377370  0.687151      0.366605
201    -1.493292     -0.842445  1.331576  0.687151      0.366605
202    -1.493292     -0.842445  1.377370  0.687151      1.129812
203    -1.493292     -0.842445  1.377370  0.687151      0.462006
204    -1.493292     -0.842445  1.377370  0.687151      0.366605

      horsepower  city-mpg  highway-mpg      price
0      0.218578 -0.675762     -0.580474  0.050618
1      0.218578 -0.675762     -0.580474  0.423708
2      1.315353 -0.982065     -0.726156  0.423708
3     -0.010980 -0.216307     -0.143429  0.107109
4      0.320603 -1.135217     -1.308883  0.541657
..      ...      ...      ...      ...
200      0.295097 -0.369459     -0.434792  0.466542
201      1.468391 -0.982065     -0.871838  0.739686
202      0.805225 -1.135217     -1.163201  1.042628
```

```

203    0.091046  0.089996   -0.580474  1.164922
204    0.295097 -0.982065   -0.871838  1.184166

```

[194 rows x 9 columns]

13.1 Merging categorical and numerical columns

```
[222]: df_new=pd.concat([df_cat,df_num],axis=1)
```

```
[223]: df_new
```

```
[223]:
```

	make	fuel-type	body-style	drive-wheels	engine-location	engine-type	\
0	0	1	0	2	0	0	
1	0	1	0	2	0	0	
2	0	1	2	2	0	5	
3	1	1	3	1	0	3	
4	1	1	3	0	0	3	
..	...	...	...	...	...	...	
200	21	1	3	2	0	3	
201	21	1	3	2	0	3	
202	21	1	3	2	0	5	
203	21	0	3	2	0	3	
204	21	1	3	2	0	3	

	symboling	normalized-losses	width	height	engine-size	\
0	1.846173	0.146152	-0.820757	-2.056808	0.104252	
1	1.846173	0.146152	-0.820757	-2.056808	0.104252	
2	0.176441	0.146152	-0.179636	-0.582442	0.628957	
3	1.011307	1.315366	0.140924	0.195695	-0.396602	
4	1.011307	1.315366	0.232512	0.195695	0.247354	
..	...	...	...	...	...	
200	-1.493292	-0.842445	1.377370	0.687151	0.366605	
201	-1.493292	-0.842445	1.331576	0.687151	0.366605	
202	-1.493292	-0.842445	1.377370	0.687151	1.129812	
203	-1.493292	-0.842445	1.377370	0.687151	0.462006	
204	-1.493292	-0.842445	1.377370	0.687151	0.366605	

	horsepower	city-mpg	highway-mpg	price
0	0.218578	-0.675762	-0.580474	0.050618
1	0.218578	-0.675762	-0.580474	0.423708
2	1.315353	-0.982065	-0.726156	0.423708
3	-0.010980	-0.216307	-0.143429	0.107109
4	0.320603	-1.135217	-1.308883	0.541657
..	...	...	...	...
200	0.295097	-0.369459	-0.434792	0.466542
201	1.468391	-0.982065	-0.871838	0.739686
202	0.805225	-1.135217	-1.163201	1.042628

203	0.091046	0.089996	-0.580474	1.164922
204	0.295097	-0.982065	-0.871838	1.184166

[194 rows x 15 columns]

[]: